Each of these three methods behaves slightly differently. The instance_double method takes a class or a string that is the name of a class. The resulting double only responds to methods that could be passed to an instance of that class. If you attempt to send the resulting double a method that is not implemented by that class, the test will error with a message that says something like "User does not implement method." The class_double method is similar but verifies against class methods on the existing class (or module), rather than instance methods. (Note that instance_double will not recognize methods defined via method_missing. When matching, instance_double uses method_defined? while class_double uses respond_to? on the class itself.)

If you want to verify against a class that has methods dynamically defined with method_missing, you can use object_double, which has an instance of the class as an argument and therefore allows RSpec to call respond_to? on that instance to determine whether the call to the stub is valid.

In addition to verifying the existence of the method, RSpec double verification ensures that the arguments passed to the method are valid. The doubled method will also have the same public/protected/private visibility as the original method.

If you use the string version of a constant name and the constant doesn't exist, RSpec will ignore the string and treat the double like a normal, unverified double. If you'd like a failure if you use a string constant name that doesn't exist, add the line mocks.verify_doubled_constant_names = true to the appropriate section in the spec_helper.rb file. (By default the section that manages mocks is wrapped in a multiline comment; you'll need to remove the comment lines.)

All three of these methods have spy forms—instance_spy, class_spy, and object_spy—which add as_null_object to the double, giving you verification that a method passed to the spy exists without having to specify a return value.

Partial Stubs

You might use a full double object to stand in for an entire object that is unavailable or prohibitively expensive to create or call in the test environment. You can also take advantage of the way Ruby allows you to open up existing classes and objects to add or override methods. It's easy to take a "real" object and stub out only the methods you need. This is extraordinarily useful when it comes to actual uses of stub objects.

In RSpec partial stubs are managed with the allow method: